node means that `head` is never `NULL`. Because `head` is a `std::unique_ptr<node>`, you need to call `head.get()` to do the comparison. Second, because the `node` now stores the data by pointer ❶, you can retrieve the pointer directly ❹ rather than having to construct a new instance of `T`. The big changes are in `push()`: you must first create a new instance of `T` on the heap and take ownership of it in a `std::shared_ptr<>` ❼ (note the use of `std::make_shared` to avoid the overhead of a second memory allocation for the reference count). The new node you create is going to be the new dummy node, so you don't need to supply the `new_value` to the constructor ❽. Instead, you set the data on the old dummy node to your newly allocated copy of the `new_value` ❾. Finally, in order to have a dummy node, you have to create it in the constructor ❷.

By now, I'm sure you're wondering what these changes buy you and how they help with making the queue thread-safe. Well, `push()` now accesses only `tail`, not `head`, which is an improvement. `try_pop()` accesses both `head` and `tail`, but `tail` is needed only for the initial comparison, so the lock is short-lived. The big gain is that the dummy node means `try_pop()` and `push()` are never operating on the same node, so you no longer need an overarching mutex. So, you can have one mutex for `head` and one for `tail`. Where do you put the locks?

You're aiming for the maximum opportunities for concurrency, so you want to hold the locks for the smallest possible length of time. `push()` is easy: the mutex needs to be locked across all accesses to `tail`, which means you lock the mutex after the new node is allocated ❽ and before you assign the data to the current tail node ❾. The lock then needs to be held until the end of the function.

`try_pop()` isn't so easy. First off, you need to lock the mutex on `head` and hold it until you're finished with `head`. In essence, this is the mutex to determine which thread does the popping, so you want to do that first. Once `head` is changed ❺, you can unlock the mutex; it doesn't need to be locked when you return the result ❻. That leaves the access to `tail` needing a